

# screenshot\_testing\_strategy

## HelixQA Screenshot Pipeline & Anti-Bluff Testing Strategy

**Version:** 1.0

**Date:** 2026-05-01

**Scope:** Full-stack analysis of current evidence collection architecture, on-demand screenshot pipeline design, client-app screenshot matrix for all platforms, anti-bluff verification methodology, and exact implementation plan.

**Based on:** Source code analysis of HelixQA repository (commit ~2026-05-01)

---

### Table of Contents

1. [Current Screenshot Architecture Deep Dive](#)
  2. [On-Demand Screenshot Requirements Analysis](#)
  3. [Client App Screenshot Matrix](#)
  4. [Anti-Bluff Testing Strategy](#)
  5. [CI/CD Integration \(No-Pipeline Model\)](#)
  6. [Exact Implementation Plan](#)
  7. [Appendix: Current Code References](#)
- 

## 1. Current Screenshot Architecture Deep Dive

### 1.1 Evidence Collection Layer (pkg/evidence)

The `Collector` is the primary centralized evidence collection hub. It is **platform-aware** but **not executor-bound** — it uses external command runners rather than the navigator's `ActionExecutor` interface.

**Core Types:**

```
type Type string
const (
    TypeScreenshot Type = "screenshot"
    TypeVideo       Type = "video"
    TypeLogcat      Type = "logcat"
    TypeStackTrace  Type = "stacktrace"
    TypeConsoleLog  Type = "console_log"
    TypeAudio       Type = "audio"
)
```

```
type Item struct {
    Type      Type      `json:"type"`
    Path      string    `json:"path"`
}
```

```

Platform  config.Platform `json:"platform"`
Step      string           `json:"step,omitempty"`
Timestamp time.Time         `json:"timestamp"`
Size      int64          `json:"size"`
}

```

### Screenshot Capture Methods per Platform:

| Platform | Method                                     | Exact Command                                                                 | Output   |
|----------|--------------------------------------------|-------------------------------------------------------------------------------|----------|
| Android  | adb shell<br>screencap<br>-p + adb<br>pull | adb shell screencap<br>-p /sdcard/helixqa-<br>screenshot.png then adb<br>pull | PNG file |
| Web      | Playwright CLI                             | npx playwright<br>screenshot --path<br><path>                                 | PNG file |
| Desktop  | ImageMagick<br>import                      | import -window root<br><path>                                                 | PNG file |

**Key Observations:** - captureAndroidScreenshot writes to /sdcard/helixqa-screenshot.png, pulls, then cleans up. - captureWebScreenshot shells out to npx playwright screenshot — **requires Playwright to be installed globally** or in a discoverable node\_modules. - captureDesktopScreenshot uses import -window root (ImageMagick) — **assumes X11 is running** and import is on PATH. - The defaultRunner returns a hardcoded error: "default runner: command execution not available in test" — this means **evidence collection is effectively disabled in pure unit tests** unless a mock CommandRunner is injected. - **Video recording** (StartRecording/StopRecording) only sets flags and allocates paths — **no actual ffmpeg or adb screenrecord process is spawned by the Collector itself**. The actual recording is delegated to the caller (autonomous pipeline) or nexus record layer. - **Audio recording** uses ffmpeg -f pulse -i <device> directly — sends SIGINT for graceful shutdown.

## 1.2 Session Recording Layer (pkg/session)

The SessionRecorder coordinates **multiple concurrent platform recordings**, maintains a timeline, and indexes screenshots.

### Architecture:

```

SessionRecorder
├── sessionID, outputDir
├── videos: map[string]*VideoManager    (per-platform video state)
├── timeline: *Timeline                 (chronological events)
└── screenshotIdx: int                  (sequential counter)

```

**VideoManager** is a **state tracker only** — it does NOT execute ffmpeg/adb directly. It tracks: - startedAt time.Time for offset calculation - recording bool for state - outputPath string for the final video file

### Screenshot Indexing:

```

func (sr *SessionRecorder) CaptureScreenshot(platform, name
    string) Screenshot {
    sr.screenshotIdx++
    path := filepath.Join(sr.outputDir, "screenshots", platform,
        fmt.Sprintf("%04d-%s.png", idx, name))
    // VideoOffset links screenshot to video timestamp
    ss := Screenshot{
        Path:      path,
        Platform:   platform,
        Name:       name,
        Index:      idx,
        Timestamp:  now,
        VideoOffset: offset,
    }
    sr.timeline.RecordEvent(TimelineEvent{...ScreenshotPath:
        path...})
    return ss
}

```

**Critical Gaps:** - CaptureScreenshot **returns a path but does NOT write bytes**. The caller (autonomous pipeline) must write the actual image data to the path. - There is **no mechanism to retrieve a screenshot by index or query** — callers must know the filesystem path. - There is **no on-demand API** — screenshots are only captured during predetermined pipeline phases. - VideoManager tracks timing but the actual video encoding is handled elsewhere (nexus record layer).

### 1.3 Platform Executor Screenshot Methods (pkg/navigator)

These are the **actual byte producers** used by the autonomous pipeline. Each implements ActionExecutor.Screenshot(ctx) ([]byte, error).

| Executor           | Platform         | Mechanism                                                                        | Validation                                                           | Ret<br>Log                |
|--------------------|------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------|
| ADBExecutor        | Android          | adb exec-out screencap<br>-p (fast path) → fallback to adb<br>shell screencap -p | Size check (<5000<br>bytes = blank);<br>isUniformImage()<br>sampling | 5<br>retri<br>500<br>dela |
| PlaywrightExecutor | Web              | Bridge script<br>{"action": "screenshot"}<br>via Node.js child process           | None in executor                                                     | No<br>retry               |
| X11Executor        | Desktop<br>Linux | import -window root<br>png: - (raw bytes to stdout)                              | None in executor                                                     | No<br>retry               |
| CLIExecutor        | CLI/TUI          | Returns<br>runner.Run(command,<br>args...) stdout as<br>“screenshot”             | None                                                                 | No<br>retry               |
| APIExecutor        | HTTP<br>APIs     | Returns empty bytes (noop)                                                       | N/A                                                                  | N/A                       |

## ADBExecutor Screenshot Validation (CRITICAL):

```
for attempt := 1; attempt <= 5; attempt++ {
    data, err := a.cmdRunner.Run(ctx, "adb", "-s", a.device,
        "exec-out", "screencap", "-p")
    if err != nil { /* fallback to shell method */ }
    if len(data) < 5000 { /* too small, retry */ }
    if isUniformImage(data) { /* blank, retry */ }
    return data, nil
}
```

**Key Observations:** - ADB screenshot uses `exec-out` for zero-copy speed (~bypasses `/sdcard I/O`). - Blank/uniform detection samples 4 bytes from the PNG payload (after header) with a threshold of 10. - Playwright screenshots go through a Node.js bridge script (`scripts/playwright-bridge.js`) that communicates via `stdin/stdout` JSON. - X11 uses `import -window root png:-` to output raw PNG bytes to `stdout`. - **CLIExecutor “screenshots” are just stdout text** — there is no visual rendering. This is a fundamental limitation for CLI/TUI visual regression.

## 1.4 Nexus Capture Layer (pkg/nexus/capture)

This is the **OCU P1/P1.5 real-time frame capture pipeline** — the most sophisticated capture layer. It implements `contracts.CaptureSource` for continuous frame streaming.

**Android** (pkg/nexus/capture/android): - `adb shell screenrecord --output-format=h264 --size WxH` - pipes H.264 NAL units over `stdout` - Splits NAL units by start-code prefix (`0x0000001`) - Emits `contracts.Frame` with `PixelFormatH264` - Kill-switches: `HELIXQA_CAPTURE_ANDROID_STUB=1` or `adb not on PATH`

**Web** (pkg/nexus/capture/web): - Uses `chromedp` (Chrome DevTools Protocol) with headless Chromium - `chromedp.CaptureScreenshot(&buf)` at configurable FPS (default 10) - Decodes PNG to BGRA8 raw pixels - Emits `contracts.Frame` with `PixelFormatBGRA8` - Kill-switches: `HELIXQA_CAPTURE_WEB_STUB=1` or no chromium on PATH

**Linux Desktop** (pkg/nexus/capture/linux): - Backend priority: `xwd + convert` (ImageMagick) → `gnome-screenshot` → `grim` (Wayland) - XWD producer captures raw X11 framebuffer, converts to PNG - Emits `contracts.Frame` with platform-detected format - Kill-switches: `HELIXQA_CAPTURE_LINUX_STUB=1`, no display env, no tools on PATH

## 1.5 Video Recording Pipeline (pkg/nexus/record)

`CaptureSource.Frames()` → `Recorder.drain()` → `FrameRing.Push()` → `Encoder.Encode()` → `WebRTCSubscriber`

**Components:** - `FrameRing`: Bounded ring buffer (default 1024 frames) for clip extraction - `Encoder`: Injectable backend — `x264`, `NVENC`, `VAAPI` stubs registered (all return `ErrNotWired` in P5) - `Clip(around time.Time, window time.Duration, out io.Writer, opts ClipOptions)`: Extracts time-windowed segments from the ring buffer - `LiveStream`: Delegates to `WebRTCSubscriber` (WHIP ingest — not wired in P5)

**Current State:** The actual video encoding to MP4 is **not yet wired in production** — all encoder backends return `ErrNotWired`. Video recording works at the capture layer (frames are produced) but the final MP4 muxing is pending P5.5.

### 1.6 Vision Server (internal/visionserver)

HTTP server (default : 8090) exposing cheaper vision analysis:

| Endpoint        | Method | Purpose                            |
|-----------------|--------|------------------------------------|
| /analyze        | POST   | Base64 PNG + prompt → VisionResult |
| /providers      | GET    | List registered vision providers   |
| /health         | GET    | Liveness probe                     |
| /learning/stats | GET    | Cache hits, request totals         |
| /learning/clear | POST   | Reset learning state               |

**Config (HELIX\_VISION\_\* env vars):** - Provider: auto (dynamic probing), qwen25vl, glm4v, etc. - Fallback chain, parallel execution, exact/differential/vector caching - Few-shot learning, change threshold (0.05)

### 1.7 Autonomous Pipeline Screenshots (pkg/autonomous)

The autonomous coordinator drives the screenshot flow during test execution:

For each test:  
  For each platform the test targets:  
    1. executor.Screenshot(testCtx) → []byte PNG  
    2. Validate: len > 0, not blank (IsBlankScreenshot)  
    3. Write to screenshotDir: "<platform>\_<testName>\_<timestamp>.png"  
    4. Store path in allScreenshots  
    5. Send to vision provider if within maxVisionScreenshots (15)

**Blank Detection (IsBlankScreenshot):** - Samples 81 points on a 9x9 grid - Computes per-channel range (max-min) across all samples - If maxRange < 20 → declared blank - Also rejects images < 1000 bytes or < 10x10 pixels

**Screenshot Downsizing:** - maxScreenshotWidth = 480px - Nearest-neighbor downscale for LLM vision API - Keeps file size under ~50KB for fast CPU inference

---

## 2. On-Demand Screenshot Requirements Analysis

### 2.1 What Currently Exists

| Feature                           | Exists? | Location                   | Limitations                             |
|-----------------------------------|---------|----------------------------|-----------------------------------------|
| Platform-aware screenshot capture | ✓       | pkg/evidence/collector.go  | Shell-out only; no executor integration |
| Per-step pre/post screenshots     | ✓       | pkg/validator/validator.go | Only during validation steps            |

| Feature                            | Exists? | Location                     | Limitations                                           |
|------------------------------------|---------|------------------------------|-------------------------------------------------------|
| Session-scoped screenshot indexing | ✓       | pkg/session/recorder.go      | Returns path; caller writes bytes                     |
| Real-time frame capture            | ✓       | pkg/nexus/capture/*          | Continuous streaming, not on-demand                   |
| Vision analysis of screenshots     | ✓       | internal/visionserver        | Requires base64 upload; no direct session integration |
| Blank/uniform detection            | ✓       | pkg/autonomous/screenshot.go | Used only in autonomous pipeline                      |
| Cross-device visual regression     | ✓       | pkg/regression/visual.go     | Requires multiple device screenshots upfront          |
| Video timeline correlation         | ✓       | pkg/session/timeline.go      | Screenshots linked to video offsets                   |

## 2.2 What's Missing for True On-Demand Capability

### Critical Gaps:

1. **No unified on-demand screenshot API:** There is no single function or endpoint that says “capture screenshot for platform X right now and give me the bytes/path”. The existing `Collector.CaptureScreenshot` is evidence-oriented (writes to disk) while `ActionExecutor.Screenshot` is executor-oriented (returns bytes). They are not unified.
2. **No programmatic screenshot retrieval:** Once a screenshot is saved, there's no API to query it by sessionID, timestamp, platform, or step name. Callers must construct filesystem paths manually.
3. **No presentational delivery format:** Screenshots are saved as raw PNG files. There's no service to serve them over HTTP, embed them in reports, or generate thumbnails.
4. **No CLI/TUI visual rendering:** `CLIExecutor` returns stdout text as “screenshot” — there is no actual visual capture of terminal state (ANSI colors, ncurses layouts, etc.).
5. **No responsive breakpoint screenshots for Web:** `PlaywrightExecutor` captures the current viewport only. There's no mechanism to capture at multiple breakpoints (mobile, tablet, desktop) in one call.
6. **No multi-monitor support for Desktop:** `X11Executor` captures `:0` (root window) only. No per-display selection, no multi-monitor stitching.
7. **No iOS support at all:** No iOS simulator or device screenshot capability exists in the codebase.
8. **No on-demand screenshot during video recording:** While `VideoManager` tracks offsets, there's no API to say “capture a frame right now from the active video stream”.

## 2.3 Proposed On-Demand Screenshot API

**Design Principles:** - **Unified interface:** One API serves all platforms - **Bytes-first:** Return [] byte by default; optionally persist to disk - **Synchronous and async:** Immediate capture for tests; background capture for sessions - **Metadata-rich:** Every screenshot includes platform, timestamp, session, step context - **Retrievable:** Query screenshots by any dimension (session, time, platform, step) - **Deliverable:** Serve via HTTP, embed in reports, generate thumbnails

### Proposed Core API:

```
// pkg/screenshot/manager.go

package screenshot

import (
    "context"
    "image"
    "time"
)

// Manager is the unified on-demand screenshot orchestrator.
type Manager struct {
    // Per-platform screenshot engines
    engines map[Platform]Engine
    // Storage backend
    store    Storage
    // Optional: link to active session recorder
    session *session.SessionRecorder
    // Vision server for analysis
    vision  visionserver.VisionExecutor
}

// Engine is the platform-specific capture implementation.
type Engine interface {
    // Capture returns raw screenshot bytes.
    Capture(ctx context.Context, opts CaptureOptions) (*Result,
        error)
    // Supported returns true if this engine is wired and
    // available.
    Supported(ctx context.Context) bool
    // Platform returns the platform identifier.
    Platform() Platform
}

// CaptureOptions parameterises a single screenshot request.
type CaptureOptions struct {
    // Format: png, jpeg, webp (default: png)
    Format string
    // Quality for lossy formats (1-100, default: 90)
}
```

```

Quality int
// Width/Height: 0 means native resolution
Width int
Height int
// FullPage: for web, capture full scrollable page
FullPage bool
// Responsive: for web, capture at multiple breakpoints
ResponsiveBreakpoints []Breakpoint
// DisplayID: for desktop, target specific display
DisplayID string
// WindowID: for desktop, target specific window
WindowID string
// WaitForRender: delay before capture (e.g. for animations)
WaitForRender time.Duration
// ValidateContent: reject blank/uniform screenshots
ValidateContent bool
// MaxRetries: number of retry attempts (default: 3)
MaxRetries int
}

// Breakpoint defines a responsive viewport size.
type Breakpoint struct {
    Name string
    Width int
    Height int
}

// Result contains a single captured screenshot.
type Result struct {
    Data []byte
    Format string
    Width int
    Height int
    Platform Platform
    Timestamp time.Time
    Duration time.Duration
    // Metadata for retrieval and linking
    SessionID string
    StepName string
    StepIndex int
    // Path is set if persisted to storage
    Path string
    // VideoOffset links to session video
    VideoOffset time.Duration
    // Thumbnail is a downscaled preview
    Thumbnail []byte
}

```



```

// Capture takes a single screenshot on the given platform.
func (m *Manager) Capture(ctx context.Context, platform
    Platform, opts CaptureOptions) (*Result, error)

// CaptureAll captures screenshots on all supported/wired
    platforms.
func (m *Manager) CaptureAll(ctx context.Context, opts
    CaptureOptions) ([]*Result, error)

// CaptureResponsive captures web screenshots at multiple
    breakpoints.
func (m *Manager) CaptureResponsive(ctx context.Context,
    breakpoints []Breakpoint, opts CaptureOptions)
    ([]*Result, error)

// CaptureMultiDisplay captures all desktop displays.
func (m *Manager) CaptureMultiDisplay(ctx context.Context, opts
    CaptureOptions) ([]*Result, error)

// Get retrieves a screenshot by its storage path or ID.
func (m *Manager) Get(ctx context.Context, id string) (*Result,
    error)

// Query retrieves screenshots matching criteria.
func (m *Manager) Query(ctx context.Context, q Query)
    ([]*Result, error)

// ServeHTTP makes screenshots available via HTTP (for reports/
    presentations).
func (m *Manager) ServeHTTP(w http.ResponseWriter, r
    *http.Request)

```

### HTTP API for On-Demand Screenshots:

```

POST /api/v1/screenshot/capture
{
  "platform": "web|android|desktop|ios|cli|tui",
  "format": "png|jpeg|webp",
  "quality": 90,
  "width": 1920,
  "height": 1080,
  "full_page": true,
  "responsive_breakpoints": [
    {"name": "mobile", "width": 375, "height": 667},
    {"name": "tablet", "width": 768, "height": 1024},
    {"name": "desktop", "width": 1920, "height": 1080}
  ],
  "validate_content": true,
  "wait_ms": 500
}

```

Response:

```
{
  "screenshots": [
    {
      "id": "ss-20260501-001",
      "platform": "web",
      "breakpoint": "desktop",
      "format": "png",
      "width": 1920,
      "height": 1080,
      "timestamp": "2026-05-01T12:00:00Z",
      "url": "/api/v1/screenshot/ss-20260501-001/download",
      "thumbnail_url": "/api/v1/screenshot/ss-20260501-001/thumbnail",
      "video_offset_ms": 12500,
      "size_bytes": 245760
    }
  ]
}
```

GET /api/v1/screenshot/{id}/download → raw image bytes  
GET /api/v1/screenshot/{id}/thumbnail → 480px wide thumbnail  
GET /api/v1/screenshot/{id}/analyze → vision analysis result  
GET /api/v1/screenshot/query?session=xxx&platform=web&step=login

**Delivery Modes:** - **Inline:** Base64 embedded in JSON response (for small thumbnails) - **URL:** HTTP URL to fetch the full image (for reports, dashboards) - **File:** Local filesystem path (for pipeline consumption) - **Stream:** WebSocket/frame channel (for live monitoring)

---

## 3. Client App Screenshot Matrix

### 3.1 Web (Browser)

**Current:** `PlaywrightExecutor.Screenshot()` via Node.js bridge → `chromedp.CaptureScreenshot()` in nexus capture.

**Strategy:**

| Aspect        | Current                   | Required                        | Implementation                                                                |
|---------------|---------------------------|---------------------------------|-------------------------------------------------------------------------------|
| Engine        | Node.js bridge + chromedp | Playwright/ CDP for consistency | Unify on chromedp or Playwright                                               |
| Viewport      | Single viewport           | Multiple responsive breakpoints | Add <code>CaptureResponsive()</code> with configurable breakpoints            |
| Full Page     | Viewport only             | Full scrollable page            | chromedp <code>fullPage</code> option; Playwright <code>fullPage: true</code> |
| Element-level | No                        | Specific DOM element            | Add <code>ElementSelector</code> to <code>CaptureOptions</code>               |
|               | No                        |                                 | Use Playwright device descriptors                                             |

| Aspect           | Current       | Required                             | Implementation                                 |
|------------------|---------------|--------------------------------------|------------------------------------------------|
| Mobile emulation |               | Device emulation (iPhone, Pixel)     |                                                |
| Dark mode        | No            | prefers-color-scheme testing         | Add ColorScheme option                         |
| Network idle     | No            | Wait for network idle before capture | Integrate with waitForLoadState('networkidle') |
| Cross-browser    | Chromium only | Firefox, WebKit                      | Add browser selection to config                |

### Proposed Implementation:

```
// WebEngine uses Playwright or chromedp for capture
type WebEngine struct {
    browserURL string
    page       *playwright.Page // or chromedp context
}

func (e *WebEngine) Capture(ctx context.Context, opts
    CaptureOptions) (*Result, error) {
    if opts.FullPage {
        // Playwright:
        page.Screenshot(playwright.PageScreenshotOptions{FullPage:
            true})
        // chromedp: chromedp.FullScreenshot
    }
    if len(opts.ResponsiveBreakpoints) > 0 {
        var results []*Result
        for _, bp := range opts.ResponsiveBreakpoints {
            e.setViewport(bp.Width, bp.Height)
            r, _ := e.captureSingle(ctx, opts)
            r.BreakpointName = bp.Name
            results = append(results, r)
        }
        return results[0], nil // or return all
    }
    return e.captureSingle(ctx, opts)
}
```

## 3.2 Desktop (Linux)

**Current:** X11Executor uses `import -window root; pkg/nexus/capture/linux`  
uses `xwd+convert` → `gnome-screenshot` → `grim`.

**Strategy:**

| Aspect         | Current                                       | Required                      | Implementation                                                                  |
|----------------|-----------------------------------------------|-------------------------------|---------------------------------------------------------------------------------|
| X11            | <code>import -window root, xwd+convert</code> | Keep as fallback              | Maintain both                                                                   |
| Wayland        | <code>grim</code>                             | Better compositor integration | Add <code>wlroots-screencopy</code> for <code>wlroots</code> -based compositors |
| Multi-monitor  | Root window only (all displays)               | Per-display capture           | Parse <code>xrandr</code> / <code>wl_output</code> to enumerate displays        |
| Window capture | No                                            | Specific window by ID/title   | <code>xwd -id &lt;windowid&gt;; screencapture -l &lt;winid&gt;</code> on macOS  |
| Cursor         | Always included                               | Optional hide/show cursor     | Add <code>CursorVisible</code> option                                           |
| HiDPI          | Not handled                                   | Scale-aware capture           | Read <code>Xft.dpi</code> / <code>GDK_SCALE</code> environment                  |

### Proposed Implementation:

```
// LinuxEngine probes available backends
type LinuxEngine struct {
    backend string // "xwd", "gnome", "grim", "pipewire"
    display string
}

func (e *LinuxEngine) Supported(ctx context.Context) bool {
    if os.Getenv("DISPLAY") != "" {
        if _, err := exec.LookPath("xwd"); err == nil {
            e.backend = "xwd"
            return true
        }
    }
    if os.Getenv("WAYLAND_DISPLAY") != "" {
        if _, err := exec.LookPath("grim"); err == nil {
            e.backend = "grim"
            return true
        }
    }
    return false
}

func (e *LinuxEngine) Capture(ctx context.Context, opts
    CaptureOptions) (*Result, error) {
    switch e.backend {
    case "xwd":
        if opts.DisplayID != "" {
```

```

        return e.captureX11Display(opts.DisplayID)
    }
    if opts.WindowID != "" {
        return e.captureX11Window(opts.WindowID)
    }
    return e.captureX11Root()
case "grim":
    return e.captureWaylandOutput(opts.DisplayID)
}
}

```

### 3.3 Desktop (macOS)

**Current:** pkg/capture/macos\_capture.go has captureMacOSScreenshot using screencapture -x.

**Strategy:**

| Aspect         | Current                            | Required                            | Implementation                        |
|----------------|------------------------------------|-------------------------------------|---------------------------------------|
| Screenshot     | screencapture -x                   | Keep; add ScreenCaptureKit for P5.5 | Use screencapture as fallback         |
| Window capture | screencapture -l <winid>           | Specific window by title/ID         | screencapture -l or AppleScript       |
| Display list   | system_profiler SPDisplaysDataType | Accurate display enumeration        | Parse JSON output                     |
| Permission     | Manual                             | Auto-request + verify               | CheckScreenRecordingPermission exists |
| Video          | GStreamer avfvideosrc              | Native ScreenCaptureKit (CGO)       | P5.5: CaptureWithScreenCaptureKit()   |

### 3.4 Desktop (Windows)

**Current:** No Windows-specific code found in the codebase.

**Strategy:** - Use PrintWindow Win32 API or Graphics.CopyFromScreen in C# bridge - For P5.5: DXGI Desktop Duplication API for video capture - Screenshot: BitBlt or DwmPrintWindow for window capture - Enumerate displays with EnumDisplayMonitors

### 3.5 Mobile (Android)

**Current:** ADBExecutor uses exec-out screencap -p; nexus capture uses adb shell screenrecord.

**Strategy:**

| Aspect           | Current                                | Required                                                   | Implementation                                      |
|------------------|----------------------------------------|------------------------------------------------------------|-----------------------------------------------------|
| Screenshot       | <code>adb exec-out screencap -p</code> | Keep; add <code>scrcpy</code> for speed                    | <code>scrcpy --no-display --record</code>           |
| Video            | <code>adb shell screenrecord</code>    | Keep; add <code>scrcpy</code> as alternative               | <code>scrcpy</code> has lower latency               |
| Multiple devices | Single device per executor             | Concurrent multi-device                                    | Device pool in Manager                              |
| Screen size/DPI  | Not handled                            | Capture at native resolution                               | <code>screencap</code> already does this            |
| Android TV       | IME handling in <code>Type()</code>    | Keep; verify screenshot timing                             | <code>DPAD_CENTER</code> before capture on slow TVs |
| Blank detection  | <code>isUniformImage()</code> sampling | Keep; add <code>IsBlankScreenshot()</code> from autonomous | Unify blank detection                               |
| Rotation         | Not handled                            | Auto-rotate screenshots                                    | Parse <code>dumpsys display</code> for orientation  |

### 3.6 Mobile (iOS)

**Current:** No iOS support exists.

**Strategy:**

| Aspect            | Status            | Implementation                                                                        |
|-------------------|-------------------|---------------------------------------------------------------------------------------|
| Simulator         | ✗ Not implemented | <code>xcrun simctl io &lt;udid&gt; screenshot &lt;path&gt;</code>                     |
| Real Device       | ✗ Not implemented | <code>ios-deploy</code> + <code>WebDriverAgent</code> (Appium) or <code>go-ios</code> |
| Video (Simulator) | ✗ Not implemented | <code>xcrun simctl io &lt;udid&gt; recordVideo &lt;path&gt;</code>                    |
| Video (Device)    | ✗ Not implemented | QuickTime + <code>avfoundation</code> or <code>ios_screen_capture</code>              |
| Frame extraction  | ✗ Not implemented | Convert simulator video to frame stream                                               |

**Priority:** iOS screenshot support is **HIGH** for comprehensive mobile coverage. Minimum viable:

```
// iOSSimulatorEngine uses xcrun simctl
type iOSSimulatorEngine struct {
    udid string
}

func (e *iOSSimulatorEngine) Capture(ctx context.Context, opts
    CaptureOptions) (*Result, error) {
```

```

    tmpFile := filepath.Join(os.TempDir(), "ios-screenshot.png")
    _, err := exec.CommandContext(ctx, "xcrun", "simctl", "io",
        e.udid, "screenshot", tmpFile).Output()
    if err != nil { return nil, err }
    data, _ := os.ReadFile(tmpFile)
    os.Remove(tmpFile)
    return &Result{Data: data, Format: "png"}, nil
}

```

### 3.7 CLI (Command Line Interface)

**Current:** `CLIExecutor.Screenshot()` returns `runner.Run(command, args...)` — i.e., the **stdout text**.

#### Strategy:

CLI “screenshots” are fundamentally different from GUI screenshots. There are two valid approaches:

**Approach A: Text-as-Evidence (Current)** - Capture stdout/stderr as text - Format as terminal transcript - No visual representation

**Approach B: Terminal Rendering (Proposed)** - Use `script` command or `asciinema` to record terminal session - Convert to image using terminal renderer (e.g., `terminalizer`, `termtosvg`, or custom HTML/CSS renderer) - Capture ANSI colors, cursor position, bold/italic formatting

#### Proposed Implementation:

```

// CLIEngine supports both text and rendered modes
type CLIEngine struct {
    command string
    args     []string
    mode     CLICaptureMode // "text" | "rendered"
}

type CLICaptureMode string
const (
    CLIModeText      CLICaptureMode = "text"
    CLIModeRendered CLICaptureMode = "rendered"
)

func (e *CLIEngine) Capture(ctx context.Context, opts
    CaptureOptions) (*Result, error) {
    if e.mode == CLIModeRendered {
        // Use asciinema or custom ANSI-to-image renderer
        return e.captureRendered(ctx, opts)
    }
    // Default: text mode
    data, err := e.runner.Run(ctx, e.command, e.args...)
    return &Result{Data: data, Format: "text/plain"}, err
}

```

```

}

func (e *CLIEngine) captureRendered(ctx context.Context, opts
    CaptureOptions) (*Result, error) {
    // 1. Run command with script/asciinema recording
    // 2. Convert recording to PNG using headless browser or
        native renderer
    // 3. Return image bytes
}

```

**Tools for Terminal Rendering:** - `asciinema`: Records terminal sessions as JSON (cast format) - `svg-term`: Converts asciinema casts to animated SVG - `termtosvg`: Records to SVG directly - `terminalizer`: Renders to GIF/PNG with custom themes - Custom: Use headless Chromium with `xterm.js` to render ANSI text to canvas

### 3.8 TUI (Terminal User Interface)

**Current:** Same as CLI — `CLIExecutor` with `stdout-as-screenshot`.

**Strategy:**

TUI apps (e.g., `htop`, `vim`, `ncdu`, `ranger`) use ncurses or similar libraries to draw on the terminal. Capturing them requires:

1. **ANSI Sequence Recording:** Record all escape sequences sent to the terminal
2. **State Reconstruction:** Reconstruct the terminal grid state from sequences
3. **Visual Rendering:** Render the grid to an image

**Implementation Options:**

| Method                 | Tool             | Output            | Notes                                                           |
|------------------------|------------------|-------------------|-----------------------------------------------------------------|
| asciinema + svg-term   | asciinema        | SVG/PNG           | Works for most TUIs; preserves colors                           |
| tmux capture-pane      | tmux             | Text              | <code>tmux capture-pane -p</code> captures current pane content |
| script + ansilove      | script, ansilove | PNG/ANSI art      | ansilove renders ANSI files to PNG                              |
| headless terminal      | xterm.js, ttyd   | PNG via Puppeteer | Most accurate; renders true terminal state                      |
| blessings/urwid direct | Python hooks     | Image             | Requires instrumentation of TUI framework                       |

**Proposed TUI Engine:**

```

// TUIEngine captures rich terminal interfaces
type TUIEngine struct {

```



```

    // Terminal emulator backend
    emulator string // "asciinema", "tmux", "xtermjs"
}

func (e *TUIEngine) Capture(ctx context.Context, opts
    CaptureOptions) (*Result, error) {
    switch e.emulator {
    case "asciinema":
        // Record for opts.WaitForRender duration, then convert
        to PNG
        return e.captureAsciinema(ctx, opts)
    case "tmux":
        // Capture current pane content
        return e.captureTmux(ctx, opts)
    case "xtermjs":
        // Launch headless browser with xterm.js, feed ANSI
        sequences, screenshot
        return e.captureXtermJS(ctx, opts)
    }
}

```

---

## 4. Anti-Bluff Testing Strategy

### 4.1 Philosophy

**“A test that passes when the feature is broken is worse than no test at all.”** —  
Constitution Article IX, HelixQA

The anti-bluff strategy ensures that every test **actually exercises the real functionality** it claims to test. The core principle is:

**TCP-open is the floor, not the ceiling. A passing `net.Dial` does not mean the service works.**

### 4.2 Test Type Matrix

HelixQA must support and validate the following test types with **100% coverage** of all supported platforms:

| Test Type   | What It Tests                     | Screenshot Role                               | Anti-Bluff Method                                   |
|-------------|-----------------------------------|-----------------------------------------------|-----------------------------------------------------|
| Unit        | Individual functions/<br>packages | None (too fast)                               | Deliberately break the<br>function; test must fail  |
| Integration | Component<br>interactions         | Per-step screenshots of<br>state transitions  | Mutate interface contract;<br>verify failure        |
| E2E         | Full user journey                 | Screenshots at every<br>action + before/after | Replace endpoint with<br>404; verify pipeline fails |
| Functional  | Feature correctness               |                                               |                                                     |

| Test Type                   | What It Tests                | Screenshot Role                            | Anti-Bluff Method                                              |
|-----------------------------|------------------------------|--------------------------------------------|----------------------------------------------------------------|
|                             |                              | Screenshot of expected UI state            | Break feature code; screenshot must show broken state          |
| <b>Security</b>             | Auth, injection, permissions | Screenshots of error pages / denied access | Bypass auth; screenshot must show login page                   |
| <b>Stress</b>               | Behavior under load          | Screenshots during peak load               | Verify UI remains responsive visually                          |
| <b>Chaos</b>                | Resilience to failures       | Screenshots during induced failures        | Kill dependency container; screenshot must show degraded state |
| <b>Benchmark</b>            | Performance baselines        | Screenshots of loading states              | Compare load-time screenshots across versions                  |
| <b>Challenge</b>            | Real-world usability         | Full session recording + screenshots       | Human-in-the-loop validation of workflow                       |
| <b>Runtime Verification</b> | Invariants during execution  | Periodic screenshots                       | Assert screenshot invariants (no crash dialogs)                |

### 4.3 Anti-Bluff Verification Methods

#### Method 1: The “Deliberately Break It” Test (CONST-035)

For every test that claims to verify feature X: 1. Run the test → must PASS 2. Introduce a deliberate bug in feature X (e.g., swap a condition, remove a handler) 3. Run the test again → must FAIL 4. If it still passes, the test is **bluffing**

**Screenshot-Specific Application:** 1. Run test that verifies “login button is visible” via screenshot analysis 2. Hide the login button in the app code (`CSS display: none` or remove element) 3. Run test again → must FAIL with screenshot showing missing button 4. If it passes, the screenshot analysis or element detection is bluffing

#### Method 2: Protocol-Layer Functional Probes

| Protocol Layer | Floor (Useless)                              | Ceiling (Real Verification)                                      |
|----------------|----------------------------------------------|------------------------------------------------------------------|
| TCP            | <code>net.Dial("tcp", host)</code> succeeds  | Actual application protocol handshake completes                  |
| HTTP           | <code>curl -I http://host</code> returns 200 | Real request with real payload returns correct body + screenshot |
| WebSocket      | <code>ws.Dial()</code> succeeds              | Real message exchange with screenshot of resulting UI state      |
| ADB            | <code>adb devices</code> lists device        | <code>adb shell screencap -p</code> returns valid PNG bytes      |
| Playwright     | Browser launches                             | Screenshot of actual page matches expected visual state          |

## Implementation:

```
// Probe verifies a screenshot pipeline actually produces images
func TestScreenshotPipelineReal(t *testing.T) {
    // Floor: executor exists
    executor := getExecutor()
    require.NotNil(t, executor)

    // Ceiling: screenshot has content
    ctx := context.Background()
    data, err := executor.Screenshot(ctx)
    require.NoError(t, err)
    require.Greater(t, len(data), 5000, "screenshot must have content")

    // Anti-bluff: decode as PNG
    img, format, err := image.Decode(bytes.NewReader(data))
    require.NoError(t, err, "must be valid image")
    require.Equal(t, "png", format)
    require.Greater(t, img.Bounds().Dx(), 100)
    require.Greater(t, img.Bounds().Dy(), 100)

    // Anti-bluff: not blank
    require.False(t, autonomous.IsBlankScreenshot(data),
        "screenshot must not be blank")
}
```

## Method 3: Real HTTP Requests with Real Responses

For web testing: - **Never** mock the HTTP client in E2E tests - Make real requests to the real server - Verify response body AND screenshot of rendered page - Compare headers, cookies, localStorage

```
func TestLoginFlowReal(t *testing.T) {
    // Real request
    resp, err := http.PostForm("http://localhost:8080/login",
        url.Values{"username": {"test"}, "password": {"test"}})
    require.NoError(t, err)
    require.Equal(t, 302, resp.StatusCode)

    // Real screenshot of resulting page
    screenshot, err := webExecutor.Screenshot(ctx)
    require.NoError(t, err)

    // Vision verification: is this the dashboard?
    result, err := vision.Analyze(screenshot, "Is this a dashboard page with a welcome message?")
    require.True(t, strings.Contains(result.Text, "yes"))
}
```

## Method 4: Real CLI Invocations with Real Output Parsing

For CLI testing: - Invoke the actual binary via `os/exec` - Capture stdout/stderr AND rendered terminal screenshot - Parse output with real string operations - Verify exit codes

```
func TestCLIOutputReal(t *testing.T) {
    cmd := exec.CommandContext(ctx, "./myapp", "status")
    out, err := cmd.CombinedOutput()
    require.NoError(t, err)
    require.Contains(t, string(out), "Status: OK")

    // Rendered screenshot of terminal
    screenshot, err := cliEngine.CaptureRendered(ctx, opts)
    require.NoError(t, err)

    // OCR or vision: does the image contain "Status: OK"?
    result, err := vision.Analyze(screenshot,
        "Does this terminal screenshot show 'Status: OK'?")
    require.Contains(t, result.Text, "yes")
}
```

## Method 5: Visual Verification (Screenshots Show Actual UI State)

The most powerful anti-bluff method. Every UI test must: 1. Take a screenshot 2. Verify the screenshot shows the **expected** state 3. If the expected element is missing, the test must fail

**Techniques:** - **Template matching:** Compare against golden reference images - **Vision LLM analysis:** Ask “Is the login button visible?” - **OCR extraction:** Read text from screenshot, verify presence - **Pixel-color assertions:** Verify specific regions have expected colors - **Structural comparison:** SSIM or perceptual hash comparison

```
func TestButtonVisible(t *testing.T) {
    screenshot, _ := executor.Screenshot(ctx)

    // Method A: Vision LLM
    result, _ := vision.Analyze(screenshot, "Is there a blue
        'Submit' button in the bottom right?")
    require.Contains(t, result.Text, "yes")

    // Method B: Template matching
    found, _ := vision.TemplateMatch(screenshot,
        submitButtonTemplate)
    require.True(t, found, "Submit button must be visible")

    // Method C: Anti-bluff – break it
    // (In a separate test file, hide the button and verify
    // failure)
}
```

## 4.4 Challenge Design

Challenges are end-to-end user journeys that prove real usability. Each challenge must:

- 1. **Define the Goal:** A concrete user task (e.g., “Log in, add item to cart, checkout”)
- 2. **Define the Success Criteria:** Observable outcomes (e.g., “Order confirmation page with order number”)
- 3. **Capture Evidence:** Full session recording + screenshots at every step
- 4. **Validate with Vision:** Use vision analysis to confirm UI states
- 5. **Measure Performance:** Track time per step, detect stagnation

**Challenge Validation Checklist:** - [ ] Challenge can be executed autonomously by HelixQA - [ ] Every step produces a screenshot showing the expected UI state - [ ] Final screenshot proves the goal was achieved - [ ] Video recording shows smooth interaction without stalls - [ ] Breaking any step in the app code causes the challenge to fail - [ ] Challenge passes on all target platforms (Web, Android, Desktop)

### Example Challenge: “Purchase Flow”

Step 1: Navigate to product page → Screenshot shows product details  
Step 2: Click "Add to Cart" → Screenshot shows cart icon with "1" badge  
Step 3: Click cart icon → Screenshot shows cart page with product  
Step 4: Click "Checkout" → Screenshot shows checkout form  
Step 5: Fill shipping info → Screenshots show typed fields  
Step 6: Click "Place Order" → Screenshot shows order confirmation with number  
Vision Validation: "Does the confirmation page show an order number?"

## 4.5 Synthetic User Workflows

Synthetic workflows are automated user journeys that run continuously to detect regressions.

**Design Principles:** 1. **Representative:** Covers the most common user paths 2. **Measurable:** Each step has timing, success/failure, screenshot 3. **Self-Validating:** No human judgment required; vision or assertion confirms success 4. **Reproducible:** Same inputs → same outputs every time 5. **Cross-Platform:** Runs on Web, Android, Desktop with platform-specific adaptations

### Workflow Types:

| Workflow        | Platforms    | Screenshots      | Validation                                 |
|-----------------|--------------|------------------|--------------------------------------------|
| Onboarding      | All          | 5-10 screenshots | Vision: “Is the welcome tutorial visible?” |
| Login/Logout    | All          | 3-5 screenshots  | Assert: profile page visible after login   |
| CRUD Operations | All          | 8-15 screenshots | Assert: created item appears in list       |
| Search          | Web, Desktop | 4-6 screenshots  | Assert: search results match query         |
| Navigation      | All          | Per-screen       | Assert: all screens reachable from menu    |

| Workflow        | Platforms    | Screenshots       | Validation                                 |
|-----------------|--------------|-------------------|--------------------------------------------|
| Error Handling  | All          | 2-4 screenshots   | Assert: error page shows helpful message   |
| Payment Flow    | Web, Android | 10-20 screenshots | Vision: “Is payment confirmed?”            |
| Settings Change | All          | 5-8 screenshots   | Assert: setting persists after app restart |

## 5. CI/CD Integration (No-Pipeline Model)

### 5.1 Current State

The repository has a “**NO CI/CD pipelines**” rule. Tests are triggered via:

```
make test          # go test ./... -count=1
make test-race     # go test ./... -race -count=1
make test-cover    # go test ./... -coverprofile=coverage.out
make vet           # go vet ./...
make lint          # golangci-lint run ./...
```

### 5.2 Test Orchestration Without CI/CD

Since there are no CI/CD pipelines, tests must be orchestrated through:

#### A. Makefile Targets (Local/Remote Execution)

```
# Heavy QA session orchestration
qa-session:
    go run ./cmd/helixqa --mode=autonomous --platforms=$
        (PLATFORMS) --output=$(HELIX_OUTPUT_DIR)

qa-screenshot-all:
    go test ./pkg/screenshot/... -run TestAllPlatforms -v

qa-challenge:
    go run ./cmd/helixqa --mode=challenge --challenge=$
        (CHALLENGE) --platforms=web,android,desktop

qa-anti-bluff:
    # Run tests, then run with deliberately broken code
    ./scripts/anti-bluff-verify.sh

qa-visual-regression:
    go test ./pkg/regression/... -run TestVisualRegression -v

qa-stress:
    go test ./pkg/... -run TestStress -count=5 -timeout=30m
```

## B. Script-Based Orchestration

```
#!/bin/bash
# scripts/run-qa-suite.sh

PLATFORMS="android,web,desktop"
OUTPUT_DIR="./qa-results/$(date +%Y%m%d_%H%M%S)"

# 1. Unit tests
make test

# 2. Screenshot pipeline verification
make qa-screenshot-all

# 3. Anti-bluff verification
make qa-anti-bluff

# 4. Challenge runs
for challenge in onboarding login checkout; do
    make qa-challenge CHALLENGE=$challenge
    # Collect evidence
    cp -r $OUTPUT_DIR/challenges/$challenge ./reports/
done

# 5. Generate report
go run ./cmd/helixqa --mode=report --input=$OUTPUT_DIR --
    formats=markdown,html
```

## C. Result Reporting

Without CI/CD, results are reported via: 1. **Local filesystem**: qa-results/session-YYYYMMDD\_HHMMSS/ 2. **Markdown/HTML reports**: Generated by HelixQA report generator 3. **JSON artifacts**: Structured data for downstream processing 4. **Screenshots directory**: screenshots/<platform>/ 5. **Video directory**: videos/<platform>-<sessionID>.mp4 6. **Timeline JSON**: timeline.json with all events

### Report Structure:

```
qa-results/
├── session-20260501_120000/
│   ├── report.md
│   ├── report.html
│   ├── timeline.json
│   └── screenshots/
│       ├── android/
│       │   ├── 0001-login_pre.png
│       │   └── 0002-login_post.png
│       ├── web/
│       │   ├── 0001-homepage.png
│       │   └── 0002-login.png
│       └── desktop/
│           └── 0001-launch.png
```

```

├── 0002-dashboard.png
├── videos/
│   ├── android-session-20260501_120000.mp4
│   ├── web-session-20260501_120000.mp4
│   └── desktop-session-20260501_120000.mp4
├── evidence/
│   ├── logcat.txt
│   └── console.log
├── tickets/
└── HELIX-001-crash-on-login.md

```

### 5.3 Heavy QA Session Orchestration

For long-running autonomous sessions:

```

// Session orchestrator coordinates multi-platform testing
type SessionOrchestrator struct {
    sessionID    string
    platforms    []Platform
    screenshot   *screenshot.Manager
    recorder     *session.SessionRecorder
    validator    *validator.Validator
    challenges   []Challenge
}

func (so *SessionOrchestrator) Run(ctx context.Context)
    (*SessionResult, error) {
    // 1. Start recording on all platforms
    for _, p := range so.platforms {
        so.recorder.StartRecording(p.String())
    }

    // 2. Run challenges
    for _, ch := range so.challenges {
        result := so.runChallenge(ctx, ch)
        // Capture per-step screenshots
        for _, step := range result.Steps {
            ss, _ := so.screenshot.Capture(ctx, ch.Platform,
                screenshot.CaptureOptions{
                    StepName: step.Name,
                    ValidateContent: true,
                })
            step.ScreenshotPath = ss.Path
        }
    }

    // 3. Stop recording
    for _, p := range so.platforms {
        so.recorder.StopRecording(p.String())
    }
}

```



```

// 4. Generate report
return so.generateReport()
}

```

---

## 6. Exact Implementation Plan

### 6.1 New Package: pkg/screenshot

#### File Structure:

```

pkg/screenshot/
├── manager.go           # Core Manager type
├── options.go           # CaptureOptions, Breakpoint, etc.
├── result.go            # Result type and helpers
├── storage.go           # Storage interface and filesystem impl
├── query.go             # Query type and filtering
├── server.go            # HTTP server for screenshot serving
├── engines/
│   ├── engine.go        # Engine interface
│   ├── web.go           # WebEngine (Playwright/chromedp)
│   ├── android.go       # AndroidEngine (ADB/scrcpy)
│   ├── ios.go           # iOSEngine (simctl)
│   ├── linux.go         # LinuxEngine (xwd/grim)
│   ├── macos.go         # macOSEngine (screencapture)
│   ├── windows.go       # WindowsEngine (Win32 API)
│   ├── cli.go           # CLIEngine (text/rendered)
│   └── tui.go           # TUIEngine (asciinema/xtermjs)
├── anti_bluff.go        # Anti-bluff verification helpers
└── anti_bluff_test.go   # Anti-bluff test suite

```

### 6.2 New Configuration Options (.env.example)

```

# — Screenshot Engine —————
HELIX_SCREENSHOT_ENGINE_WEB=playwright           # playwright |
chromedp
HELIX_SCREENSHOT_ENGINE_ANDROID=adb              # adb | scrcpy
HELIX_SCREENSHOT_ENGINE_DESKTOP=auto             # auto | xwd |
grim | screencapture
HELIX_SCREENSHOT_ENGINE_IOS=simctl               # simctl | go-ios
HELIX_SCREENSHOT_ENGINE_CLI=text                 # text | rendered
HELIX_SCREENSHOT_ENGINE_TUI=asciinema            # asciinema |
tmux | xtermjs

# — On-Demand Screenshot Server —————
HELIX_SCREENSHOT_SERVER_ENABLED=true
HELIX_SCREENSHOT_SERVER_ADDR=:8091
HELIX_SCREENSHOT_SERVER_THUMBNAIL_WIDTH=480
HELIX_SCREENSHOT_SERVER_MAX_UPLOAD_MB=50

```

```

# — Responsive Breakpoints —
HELIX_SCREENSHOT_BREAKPOINTS=mobile:375x667,tablet:768x1024,desktop:1920x1080

# — Screenshot Validation —
HELIX_SCREENSHOT_VALIDATE_CONTENT=true
HELIX_SCREENSHOT_MIN_SIZE_BYTES=5000
HELIX_SCREENSHOT_MAX_RETRIES=3
HELIX_SCREENSHOT_RETRY_DELAY_MS=500

# — Storage —
HELIX_SCREENSHOT_STORAGE=filesystem
    # filesystem | s3 | memory
HELIX_SCREENSHOT_STORAGE_PATH=./qa-results/screenshots
HELIX_SCREENSHOT_KEEP_DAYS=30

# — iOS / macOS —
HELIX_IOS_SIMULATOR_UDID=                # auto-detect if available
HELIX_MACOS_PERMISSION_AUTO_REQUEST=true

```

### 6.3 New CLI Flags (cmd/helixqa)

```

helixqa screenshot capture \
  --platform=web \
  --format=png \
  --width=1920 \
  --height=1080 \
  --full-page \
  --output=./screenshot.png

helixqa screenshot capture-all \
  --platforms=web,android,desktop \
  --responsive \
  --output-dir=./screenshots/

helixqa screenshot serve \
  --addr=:8091 \
  --storage=./qa-results/screenshots

helixqa screenshot query \
  --session=session-20260501_120000 \
  --platform=web \
  --step=login

helixqa screenshot anti-bluff \
  --test=TestLoginFlow \
  --break-method=hide_element \
  --element="#login-button"

```

## 6.4 New APIs to Add

### A. Manager API (pkg/screenshot/manager.go)

```
func NewManager(opts ...ManagerOption) *Manager
func (m *Manager) RegisterEngine(e Engine) error
func (m *Manager) Capture(ctx context.Context, platform
    Platform, opts CaptureOptions) (*Result, error)
func (m *Manager) CaptureAll(ctx context.Context, opts
    CaptureOptions) ([]*Result, error)
func (m *Manager) CaptureResponsive(ctx context.Context, url
    string, bps []Breakpoint) ([]*Result, error)
func (m *Manager) Get(ctx context.Context, id string) (*Result,
    error)
func (m *Manager) Query(ctx context.Context, q Query)
    ([]*Result, error)
func (m *Manager) Delete(ctx context.Context, id string) error
func (m *Manager) Analyze(ctx context.Context, id string, prompt
    string) (*vision.VisionResult, error)
```

### B. Storage Interface (pkg/screenshot/storage.go)

```
type Storage interface {
    Store(ctx context.Context, r *Result) (string, error)
    Retrieve(ctx context.Context, id string) (*Result, error)
    Query(ctx context.Context, q Query) ([]*Result, error)
    Delete(ctx context.Context, id string) error
    Thumbnail(ctx context.Context, id string, maxWidth int)
        ([]byte, error)
}
```

### C. HTTP Server (pkg/screenshot/server.go)

```
func NewServer(m *Manager, cfg ServerConfig) *Server
// Routes:
// POST /api/v1/screenshot/capture
// GET  /api/v1/screenshot/{id}
// GET  /api/v1/screenshot/{id}/download
// GET  /api/v1/screenshot/{id}/thumbnail
// POST /api/v1/screenshot/{id}/analyze
// GET  /api/v1/screenshot/query
// DELETE /api/v1/screenshot/{id}
```

## 6.5 Modifications to Existing Code

A. pkg/evidence/collector.go - Deprecate CaptureScreenshot in favor of pkg/screenshot.Manager - Add WithManager(m \*screenshot.Manager) option - Delegate screenshot capture to Manager while maintaining backward compatibility

**B.** pkg/session/recorder.go - Add ScreenshotManager  
\*screenshot.Manager field - CaptureScreenshot should call manager.Capture() and write bytes to the returned path - Add GetScreenshot(index int) (\*screenshot.Result, error) for retrieval

**C.** pkg/validator/validator.go - Replace ScreenshotFunc with \*screenshot.Manager - ValidateStep should use manager.Capture() for pre/post screenshots - Add option to run vision analysis on post-screenshot for automatic validation

**D.** pkg/autonomous/pipeline.go - Replace manual executor.Screenshot() calls with screenshot.Manager.Capture() - Add CaptureOptions per test step for platform-specific options - Integrate responsive breakpoints for web tests

**E.** pkg/navigator/executor.go - Add Platform() Platform method to ActionExecutor interface - Consider deprecating Screenshot() in favor of screenshot engines - Keep for backward compatibility during transition

## 6.6 Anti-Bluff Test Implementation

**File:** pkg/screenshot/anti\_bluff\_test.go

```
package screenshot

import (
    "context"
    "testing"
    "time"
)

// TestScreenshotNotBlank verifies that screenshots contain
// actual content.
func TestScreenshotNotBlank(t *testing.T) {
    engines := []struct {
        name    string
        engine  Engine
    }{
        {"web", newWebEngine(t)},
        {"android", newAndroidEngine(t)},
        {"desktop", newLinuxEngine(t)},
    }

    for _, e := range engines {
        t.Run(e.name, func(t *testing.T) {
            ctx := context.Background()
            result, err := e.engine.Capture(ctx, CaptureOptions{
                ValidateContent: true,
                WaitForRender:    1 * time.Second,
            })
            if err != nil {
                t.Skipf("engine not wired: %v", err)
            }
        })
    }
}
```

```

        // 1. Must have data
        if len(result.Data) < 5000 {
            t.Fatalf("screenshot too small: %d bytes",
len(result.Data))
        }

        // 2. Must decode as image
        img, format, err :=
image.Decode(bytes.NewReader(result.Data))
        if err != nil {
            t.Fatalf("invalid image: %v", err)
        }
        if format != "png" && format != "jpeg" {
            t.Fatalf("unexpected format: %s", format)
        }

        // 3. Must have reasonable dimensions
        if img.Bounds().Dx() < 100 || img.Bounds().Dy() <
100 {
            t.Fatalf("image too small: %dx%d",
img.Bounds().Dx(), img.Bounds().Dy())
        }

        // 4. Must not be blank
        if autonomous.IsBlankScreenshot(result.Data) {
            t.Fatal("screenshot is blank/uniform")
        }
    })
}

// TestDeliberatelyBreakScreenshot verifies anti-bluff by hiding
// elements.
func TestDeliberatelyBreakScreenshot(t *testing.T) {
    if os.Getenv("HELIX_ANTIBLUFF_BREAK") != "1" {
        t.Skip("Set HELIX_ANTIBLUFF_BREAK=1 to run deliberate-
break tests")
    }

    // This test runs against a modified app where the login
    // button is hidden.
    // The screenshot test MUST fail.
    ctx := context.Background()
    result, err := webEngine.Capture(ctx, CaptureOptions{
        URL: "http://localhost:8080/login",
    })
    require.NoError(t, err)

```

```

// Vision analysis: is login button visible?
analysis, err := vision.Analyze(result.Data, "Is the login
    button visible?")
require.NoError(t, err)

// When HELIX_ANTIBLUFF_BREAK=1, the button is hidden, so
    analysis should say "no"
if strings.Contains(strings.ToLower(analysis.Text), "yes") {
    t.Fatal("ANTI-BLUFF FAILURE: test passed when button was
        deliberately hidden")
}
}

```

## 6.7 Implementation Phases

| Phase       | Deliverables                                                                                             | Timeline | Risk                   |
|-------------|----------------------------------------------------------------------------------------------------------|----------|------------------------|
| <b>P1</b>   | Core pkg/screenshot package; Manager, Result, Storage interfaces; filesystem storage                     | Week 1   | Low                    |
| <b>P2</b>   | WebEngine + AndroidEngine + LinuxEngine with Capture() methods; HTTP server with / capture and /download | Week 2   | Medium                 |
| <b>P3</b>   | iOSEngine + macOSEngine + CLIEngine + TUIEngine; responsive breakpoints; multi-display support           | Week 3   | High (iOS permissions) |
| <b>P4</b>   | Integration with pkg/session, pkg/validator, pkg/autonomous; on-demand API in cmd/helixqa                | Week 4   | Medium                 |
| <b>P5</b>   | Anti-bluff test suite; challenge screenshot validation; visual regression integration                    | Week 5   | Low                    |
| <b>P5.5</b> | Native video frame extraction from screenshots; NVENC/VAAPI encoder wiring; ScreenCaptureKit on macOS    | Week 6-8 | High (CGO, hardware)   |

## 7. Appendix: Current Code References

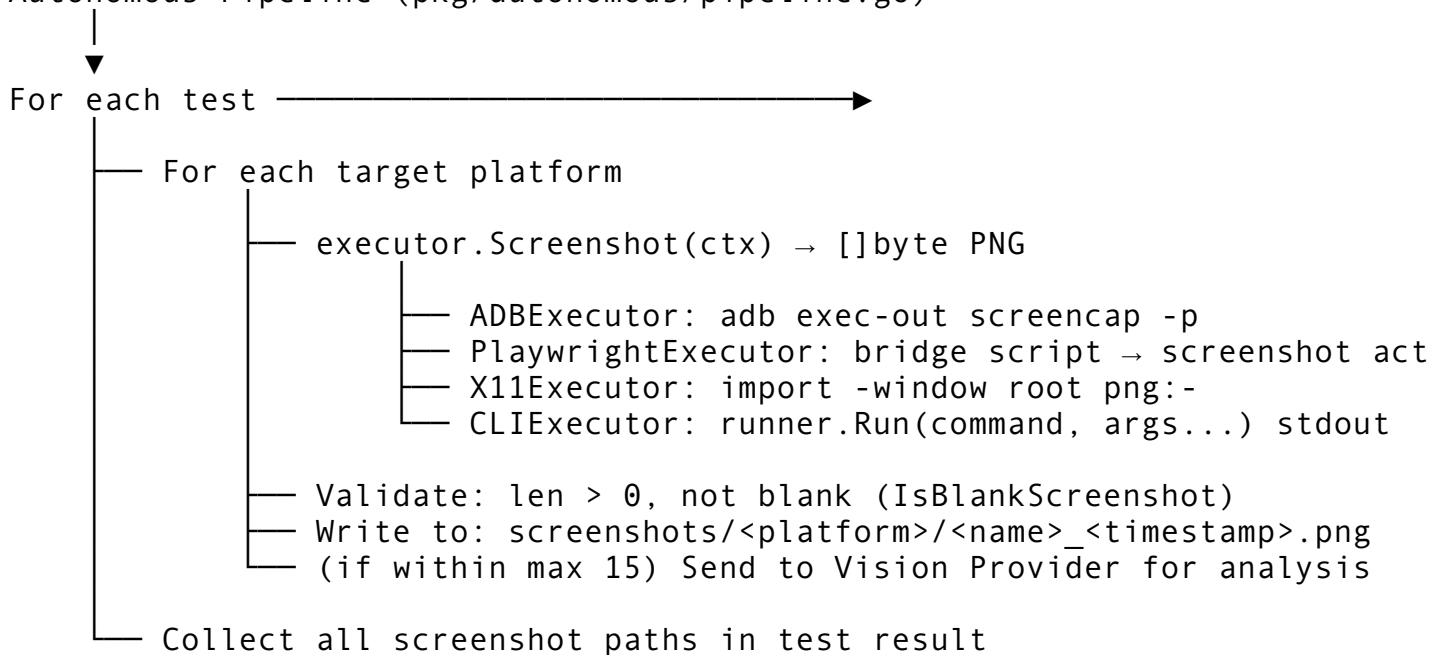
### 7.1 Key Source Files

| File                                      | Purpose                                      | Lines |
|-------------------------------------------|----------------------------------------------|-------|
| pkg/evidence/<br>collector.go             | Evidence collection hub                      | 521   |
| pkg/session/<br>recorder.go               | Session recording coordinator                | 247   |
| pkg/session/<br>timeline.go               | Timeline event tracking                      | 152   |
| pkg/session/video.go                      | Video state tracking (no<br>encoding)        | 111   |
| pkg/validator/<br>validator.go            | Step validation with pre/post<br>screenshots | 262   |
| pkg/navigator/<br>executor.go             | ActionExecutor interface +<br>ADBExecutor    | 424   |
| pkg/navigator/<br>playwright_executor.go  | Playwright web executor                      | 340   |
| pkg/navigator/<br>x11_executor.go         | X11 desktop executor                         | 169   |
| pkg/navigator/<br>cli_executor.go         | CLI/TUI executor                             | 135   |
| pkg/navigator/<br>api_executor.go         | HTTP API executor                            | 133   |
| pkg/nexus/capture/<br>android/source.go   | ADB H.264 frame capture                      | 307   |
| pkg/nexus/capture/<br>linux/source.go     | Linux screenshot frame capture               | 221   |
| pkg/nexus/capture/web/<br>source.go       | Chromedp web frame capture                   | 290   |
| pkg/nexus/record/<br>recorder.go          | Frame recording pipeline                     | 220   |
| pkg/nexus/native/<br>contracts/capture.go | Capture interface contracts                  | 137   |
| pkg/nexus/native/<br>contracts/record.go  | Recorder interface contracts                 | 49    |
| pkg/autonomous/<br>screenshot.go          | Blank detection + resizing                   | 153   |
| pkg/autonomous/<br>pipeline.go            | Autonomous QA pipeline<br>(screenshot flow)  | ~2500 |
| pkg/regression/<br>visual.go              | Cross-device visual regression               | 460   |

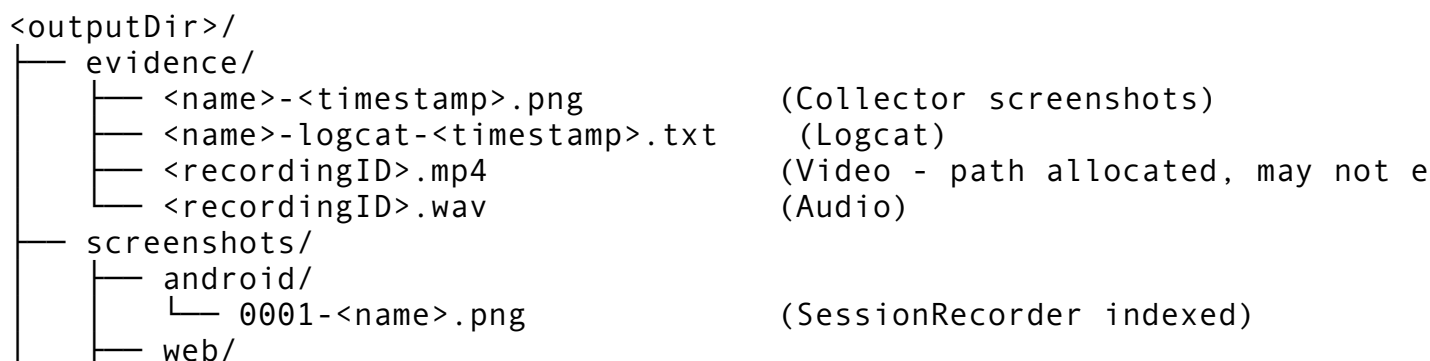
| File                              | Purpose                            | Lines |
|-----------------------------------|------------------------------------|-------|
| pkg/visual/comparator.go          | Screenshot comparison tools        | ~400  |
| pkg/capture/macos_capture.go      | macOS capture (build-tagged)       | 341   |
| internal/visionserver/server.go   | Vision HTTP server                 | 73    |
| internal/visionserver/handlers.go | Vision API handlers                | 199   |
| internal/visionserver/config.go   | Vision server config               | 190   |
| .env.example                      | Environment configuration template | 137   |
| Makefile                          | Build and test targets             | 48    |

## 7.2 Current Screenshot Flow During Autonomous Session

Autonomous Pipeline (pkg/autonomous/pipeline.go)



## 7.3 Evidence Storage Layout (Current)





```

├── 0001-<name>.png
├── desktop/
│   └── 0001-<name>.png
├── videos/
│   ├── android-<sessionID>.mp4
│   ├── web-<sessionID>.mp4
│   └── desktop-<sessionID>.mp4
└── timeline.json

```

## 7.4 Configuration Parameters Affecting Screenshots

| Env Var                           | Default               | Effect                            |
|-----------------------------------|-----------------------|-----------------------------------|
| HELIX_RECORDING_SCREENSHOTS       | true                  | Enable/disable screenshot capture |
| HELIX_RECORDING_SCREENSHOT_FORMAT | png                   | Screenshot file format            |
| HELIX_RECORDING_VIDEO             | true                  | Enable/disable video recording    |
| HELIX_RECORDING_VIDEO_QUALITY     | medium                | Video quality setting             |
| HELIX_RECORDING_FFMPEG_PATH       | /usr/bin/ffmpeg       | ffmpeg binary location            |
| HELIX_OUTPUT_DIR                  | ./qa-results          | Base output directory             |
| HELIX_ANDROID_DEVICE              | emulator-5554         | ADB target device                 |
| HELIX_WEB_URL                     | http://localhost:8080 | Web target URL                    |
| HELIX_DESKTOP_DISPLAY             | :0                    | X11 display                       |
| HELIXQA_CAPTURE_ANDROID_STUB      | 0                     | Disable real ADB capture          |
| HELIXQA_CAPTURE_LINUX_STUB        | 0                     | Disable real Linux capture        |
| HELIXQA_CAPTURE_WEB_STUB          | 0                     | Disable real web capture          |

| Env Var            | Default                                                                                                                                                                                                                         | Effect                                |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| HELIXQA_ADB_SERIAL | `  ADB device serial<br>override    <br>HELIX_VISION_PROVIDER auto <br>Vision provider selection  <br> <br>HELIX_VISION_MAX_IMAGE_SIZE <br>4096  Max image dimension<br>for vision    <br>HELIX_VISION_SSIM_THRESHOLD <br>0.95` | Screenshot<br>similarity<br>threshold |

## Summary

This strategy document provides a comprehensive analysis of HelixQA's current screenshot and evidence collection architecture, identifies critical gaps for on-demand screenshot capability, defines platform-specific strategies for all client app types (Web, Desktop Linux/macOS/Windows, Android, iOS, CLI, TUI), designs a rigorous anti-bluff testing methodology, outlines test orchestration without CI/CD pipelines, and delivers an exact implementation plan with file paths, function signatures, and phased delivery schedule.

**Top Priority Actions:** 1. Create `pkg/screenshot` package with unified Manager and Engine interfaces 2. Implement on-demand HTTP API for screenshot capture, retrieval, and analysis 3. Add iOS simulator screenshot engine (currently completely absent) 4. Add TUI/CLI rendered screenshot capability (ascinema/xtermjs) 5. Integrate Manager with `pkg/session`, `pkg/validator`, and `pkg/autonomous` 6. Implement anti-bluff test suite that deliberately breaks features to verify test validity 7. Add responsive breakpoint capture for web platforms 8. Add multi-display capture for desktop platforms

**Immediate Code Changes:** - Add `Platform()` `Platform` to `ActionExecutor` interface - Add `screenshot.Manager` reference to `SessionRecorder` - Create `screenshot.CaptureOptions` with `WaitForRender`, `ValidateContent`, `MaxRetries` - Implement `WebEngine` wrapping `PlaywrightExecutor` with full-page and responsive support - Create HTTP handlers for `/api/v1/screenshot/*` endpoints - Add anti-bluff environment variable `HELIX_ANTIBLUFF_BREAK` for deliberate-break tests